

การสร้างชุดข้อมูลข้ามแพลตฟอร์มสำหรับการตรวจจับมัลแวร์
ด้วยการเรียนรู้ของเครื่องจากบันทึกเหตุการณ์ Sysmon
บนระบบปฏิบัติการ Linux และ Windows
A CROSS-PLATFORM SYSMON-BASED DATASET
FOR MACHINE-LEARNING MALWARE DETECTION
ON LINUX AND WINDOWS

บทคัดย่อ

บทความนี้นำเสนอการสร้างและประเมินชุดข้อมูลบันทึกเหตุการณ์ Sysmon จาก Linux และ Windows สำหรับศึกษาการตรวจจับมัลแวร์ด้วยการเรียนรู้ของเครื่อง การทดลองใช้ Atomic Red Team ร่วมกับโปรแกรมทดลองสร้างหากลุ่มสถานการณ์โจมตีและพฤติกรรมปกติ และกำหนดป้ายกำกับสถานการณ์โจมตีด้วยการสืบสายกระบวนการ ชุดข้อมูลประกอบด้วย 81,331 เหตุการณ์ 80 คอลัมน์ และมีเหตุการณ์ที่กำหนดป้ายกำกับอันตราย 33.8% เมื่อนำเหตุการณ์มารวมเป็นคุณลักษณะระดับกระบวนการและตัดคุณลักษณะที่เกี่ยวข้องกับขึ้นเติมข้อมูลออก เหลือคุณลักษณะ 32 ตัว ผลเบื้องต้นจากการแบ่ง holdout ตาม ProcessGuid ด้วย GroupShuffleSplit สัดส่วน 70:30 พบว่า Random Forest ให้ Accuracy 94.9%, F1 0.924 และ ROC-AUC 0.988 ส่วน Local Outlier Factor ให้ F1 0.714 และ ROC-AUC 0.849 การประเมินเพิ่มเติมโดยกันคู่แพลตฟอร์ม-สถานการณ์ออกทีละกลุ่มให้ F1 เฉลี่ยของ Random Forest เท่ากับ 0.764 ผลสะท้อนความสามารถในการจำแนกภายในแล็บ และแสดงความจำเป็นของการระบุหน่วยตัวอย่าง วิธีแบ่งข้อมูล และขอบเขตการประเมินอย่างชัดเจน การยืนยันผลภายนอกแล็บยังต้องเพิ่มรอบเก็บ เครื่องเป้าหมาย และการทดสอบกับมัลแวร์จริง

คำสำคัญ – Sysmon, การตรวจจับมัลแวร์, ชุดข้อมูลข้ามแพลตฟอร์ม, คุณลักษณะระดับกระบวนการ, การประเมินแบบจัดกลุ่ม

ABSTRACT

This paper presents a cross-platform Sysmon dataset for studying machine-learning malware detection on Linux and Windows. Atomic Red Team and controlled executable workloads generate five attack scenario categories alongside benign activity. Attack-session labels are assigned through process lineage. The dataset contains 81,331 events and 80 columns, with 33.8% of events labeled malicious. Process aggregation and removal of enrichment-related features produce 32 features for classification. In a single 70:30 process holdout using GroupShuffleSplit, preliminary results show that Random Forest achieves 94.9% accuracy, an F1 score of 0.924, and a ROC-AUC of 0.988. Local Outlier Factor achieves an F1 score of 0.714 and a ROC-AUC of 0.849. Additional holdouts of individual platform-scenario combinations yield a mean Random Forest F1 score of 0.764. These results characterize performance within the laboratory and highlight the need to report the sample unit, split protocol, and evaluation scope. Further validation requires independent collection runs, additional hosts, and real malware samples.

Keywords – Sysmon, malware detection, cross-platform dataset, process-level features, grouped evaluation

1. บทนำ

การตรวจจับมัลแวร์เชิงพฤติกรรมอาศัยการสังเกตกิจกรรมขณะโปรแกรมทำงาน เช่น การสร้างกระบวนการ การเขียนไฟล์ และการติดต่อเครือข่าย บันทึกจาก System Monitor หรือ Sysmon ช่วยรวบรวมกิจกรรมเหล่านี้พร้อมข้อมูลของกระบวนการที่เกี่ยวข้อง [1] อย่างไรก็ตาม เหตุการณ์เพียงรายการเดียวอาจเกิดได้ทั้งจากโปรแกรมปกติและโปรแกรมที่เกี่ยวข้องกับการโจมตี การรวมเหตุการณ์ของกระบวนการจึงเป็นแนวทางสำหรับศึกษารูปแบบการทำงานตลอดช่วงเวลาที่เกี่ยวข้อง

งานนี้วิเคราะห์พฤติกรรมแบบ host-based คือเก็บเหตุการณ์จาก Sysmon ที่ติดตั้งบนเครื่องเป้าหมาย แล้วเชื่อมโยงกิจกรรมกับกระบวนการบนเครื่องนั้น EventID 1 ให้ข้อมูลการเริ่มกระบวนการและคำสั่ง EventID 11 และ 23 ให้ร่องรอยการสร้างหรือเขียนทับไฟล์และการลบไฟล์ ส่วน EventID 12 และ 13 ให้กิจกรรม registry บน Windows แม้ EventID 3 เป็นการเชื่อมต่อเครือข่าย ก็ยังเป็นข้อมูลจากโฮสต์ เพราะบันทึกกระบวนการต้นทางร่วมกับ IP และพอร์ต [1] จึงศึกษาการติดต่อเครือข่ายร่วมกับกิจกรรมของโปรแกรมบนเครื่องได้

การสร้างชุดข้อมูลสำหรับงานนี้ต้องพิจารณาทั้งความแตกต่างของระบบปฏิบัติการและความสัมพันธ์ระหว่างตัวอย่าง Windows และ Linux มีรูปแบบคำสั่ง เส้นทางไฟล์ และการรองรับเหตุการณ์ต่างกัน ขณะเดียวกัน เหตุการณ์จากกระบวนการหรือรอบทดลองเดียวกันอาจมีลักษณะคล้ายกันมาก หากแบ่งตัวอย่างโดยไม่คำนึงถึงความสัมพันธ์เหล่านี้ ผลทดสอบอาจสะท้อนการจดจำสภาพแวดล้อมมากกว่าความสามารถในการจำแนกพฤติกรรม ประเด็นดังกล่าวเกี่ยวข้องกับอคติและการรั่วไหลของข้อมูลในงานด้านความมั่นคงปลอดภัยเชิงเรียนรู้ให้มีความสำคัญ [2], [3]

งานวิจัยนี้ตั้งโจทย์ว่าบันทึก Sysmon จากสองแพลตฟอร์มสามารถนำมารวมเป็นชุดข้อมูลระดับกระบวนการเพื่อศึกษาการจำแนกกิจกรรมปกติและกิจกรรมที่เกี่ยวข้องกับการโจมตีได้อย่างไร และผลการประเมินเปลี่ยนแปลงอย่างไรเมื่อเปลี่ยนวิธีแบ่งข้อมูล การทดลองจัดสถานการณ์ตามกลุ่มพฤติกรรมในกรอบ MITRE ATT&CK [4] และใช้ Atomic Red Team [5] ร่วมกับโปรแกรมทดลองภายใต้แล็บที่ควบคุมการเก็บบันทึกและการกำหนดป้ายกำกับ

ส่วนสนับสนุนของงานประกอบด้วยการสร้างชุดข้อมูล Sysmon จาก Linux และ Windows ภายใต้ประเภทสถานการณ์ร่วมกัน การกำหนดป้ายกำกับโดยเชื่อมโยงกระบวนการที่เริ่มจากชุดทดสอบกับลูกหลาน การสร้างสถิติระดับกระบวนการที่รวมกิจกรรมหลายเหตุการณ์ และการประเมินแบบจำลองภายใต้การแบ่งตามกระบวนการและการกันคู่แพลตฟอร์ม-สถานการณ์ออก ผลที่รายงานใช้ระบุสมรรถนะและข้อจำกัดของชุดข้อมูลภายในแล็บ เพื่อเป็นพื้นฐานสำหรับการเก็บและประเมินข้อมูลรอบถัดไป

2. งานที่เกี่ยวข้อง

2.1 บันทึก Sysmon และข้อมูลข้ามแพลตฟอร์ม

Mahmoud และคณะ [6] ศึกษาการวิเคราะห์มัลแวร์ด้วย Sysmon ร่วมกับระบบ ELK เพื่อรวบรวมและวิเคราะห์กิจกรรมบนเครื่อง ส่วน BarongTrace [7] นำเสนอชุดข้อมูลเหตุการณ์มัลแวร์บน Linux งานทั้งสองแสดงแนวทางการใช้บันทึกที่ระดับโฮสต์ในสภาพแวดล้อมต่างกัน สำหรับงานนี้ บันทึกจากแต่ละแพลตฟอร์มถูกแปลงเป็นตารางร่วมกัน โดยยังเก็บรหัสแพลตฟอร์ม เครื่อง และรอบทดลองไว้สำหรับตรวจสอบย้อนกลับ

2.2 การวิเคราะห์พฤติกรรมและหน่วยกระบวนการ

Sgandurra และคณะ [8] ศึกษาการตรวจจับ ransomware จากพฤติกรรมที่เก็บระหว่างการทำงานของโปรแกรม และ Herrera-Silva กับ Hernandez-Alvarez [9] นำเสนอชุดข้อมูลคุณลักษณะเชิงพลวัตสำหรับ ransomware ส่วน Pratomo และคณะ [10] ศึกษาการวิเคราะห์มัลแวร์ในระดับเครื่องและระดับกระบวนการ งานนี้ใช้กระบวนการเป็นหน่วยวิเคราะห์เพื่อเชื่อมโยงจำนวนเหตุการณ์ กิจกรรมไฟล์ เครือข่าย และช่วงเวลาการทำงานเข้าด้วยกัน แทนการตัดสินใจจากเหตุการณ์รายการเดียว

2.3 ข้อมูลสืบสายและความน่าเชื่อถือของการประเมิน

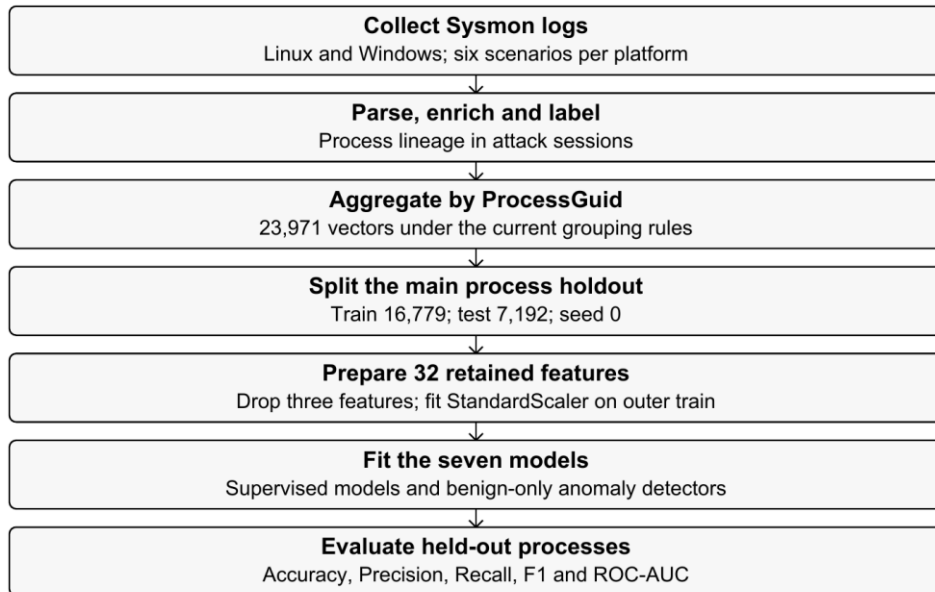
UNICORN [11] และ Kairos [12] ใช้ข้อมูล provenance เพื่อวิเคราะห์ความสัมพันธ์ระหว่างกิจกรรมในระบบ งานนี้ใช้ความสัมพันธ์ของกระบวนการแม่และลูกในขั้นกำหนดป้ายกำกับเพื่อระบุว่ากิจกรรมใดเชื่อมโยงกับกระบวนการเริ่มต้นของการทดลอง ขณะเดียวกัน ข้อเสนอแนะเรื่องอคติในการประเมิน [2],

[3] เป็นเหตุผลให้รายงานวิธีแบ่งข้อมูลอย่างชัดเจน และแยกการทดสอบกระบวนการที่กันออกจากการทดสอบรอบเก็บหรือเครื่องที่ไม่เคยใช้ในการฝึก

กำหนดป้ายกำกับ และสร้างคุณลักษณะระดับกระบวนการ ก่อนแบ่งชุดข้อมูลและฝึกแบบจำลอง ภาพ 1 แสดงลำดับของการทดลอง holdout หลัก การประเมินแบบกันคู่แพลตฟอร์ม-สถานการณ์ออกดำเนินแยกตามกลุ่มที่ระบุในหัวข้อ 3.6

3. ระเบียบวิธีวิจัย

กระบวนการทดลองเริ่มจากการจำลองกิจกรรมและเก็บบันทึก Sysmon จากนั้นแยกวิเคราะห์ เติมข้อมูลกระบวนการ



ภาพ 1 กระบวนการเตรียมข้อมูลและประเมินด้วย holdout ระดับกระบวนการ

3.1 สภาพแวดล้อมและการเก็บบันทึก

การทดลองใช้ VMware Workstation และ Vagrant จัดการเครื่องเสมือน Linux และ Windows บนโฮสต์ Windows 11 เครื่อง Linux ใช้ Ubuntu 22.04 และ Sysmon for Linux โดยส่งบันทึกผ่าน journald และ rsyslog ด้วย TCP ไปยังโฮสต์ ส่วน Windows เก็บบันทึกจาก Sysmon และส่งออกหลังจบสถานการณ์ เครื่องเสมือนทำงานที่ละเครื่อง และคืนสภาพ snapshot ก่อนเริ่มสถานการณ์เพื่อควบคุมสภาพแวดล้อม

กิจกรรมปกติและกิจกรรมโจมตีของแต่ละแพลตฟอร์มเก็บบนเครื่องเป้าหมายเดียวกัน การตั้งค่าแล็บฝั่ง Windows ปิด Defender เพื่อให้ชุดทดสอบทำงานได้ จึงต้องพิจารณาการตั้งค่านี้เมื่อประเมินการนำผลไปใช้ในระบบจริง ข้อมูลปัจจุบันมีหนึ่งรอบเก็บต่อสถานการณ์ต่อแพลตฟอร์ม รวม 12 รอบ รหัสรอบทดลองถูกเก็บไว้ในคอลัมน์ run_id

3.2 เหตุผลเลือกสถานการณ์และพฤติกรรมบนโฮสต์

แต่ละแพลตฟอร์มมีพฤติกรรมปกติและทำกลุ่มสถานการณ์โจมตี ได้แก่ ransomware, trojan/backdoor, botnet, cryptominer และ exploitation ฝั่ง Linux ใช้ทั้ง Atomic Red Team และโปรแกรมทดลอง ได้แก่ XMIRig, backdoor และชุดทดสอบยกระดับสิทธิ์ ส่วน Windows ใช้ Atomic Red Team ร่วมกับกิจกรรมไฟล์ งานคำนวณ และการติดต่อเครือข่ายในแล็บ เซิร์ฟเวอร์ C2 และ pool อยู่ภายในสภาพแวดล้อมทดลอง ซึ่งกลุ่มเหล่านี้ใช้จัดสถานการณ์และไม่ได้ระบุว่ามีตัวอย่างมัลแวร์จากธรรมชาติที่หาตระกูล

เลือกทำกลุ่มเพื่อให้ชุดข้อมูลมีพฤติกรรมบนโฮสต์หลายด้านที่บันทึก Sysmon สังเกตได้ ได้แก่ ผลกระทบต่อไฟล์ การคงอยู่ในระบบ การสื่อสารกับผู้ควบคุม บริบทการใช้ทรัพยากร และการยกระดับสิทธิ์ อีกทั้งสามารถจัดกิจกรรมที่มีวัตถุประสงค์ใกล้เคียงกันบน Linux และ Windows โดยอิงเทคนิค MITRE ATT&CK [4]

และชุดทดสอบ Atomic Red Team [5] การเลือกนี้กำหนดขอบเขตการทดลองเชิงปฏิบัติการ ไม่ได้อ้างว่าหากกลุ่มเป็นมัลแวร์ที่พบบ่อยที่สุดหรือครอบคลุมภัยคุกคามทั้งหมด

Ransomware ใช้ศึกษากิจกรรมที่กระทบไฟล์จำนวนมาก สคริปต์สร้างไฟล์เป้าหมาย เรียกชุดทดสอบเข้ารหัส และสร้างหรือลบไฟล์ในพื้นที่ทดลอง ร่องรอยที่ศึกษาได้คือกระบวนการที่เรียกเครื่องมือ จำนวนไฟล์ที่เกี่ยวข้อง และสัดส่วน FileCreate กับ FileDelete สัญญาณเหล่านี้สะท้อนกิจกรรมไฟล์ แต่บันทึกดังกล่าวเพียงอย่างเดียวไม่ได้ยืนยันว่าเนื้อหาทุกไฟล์ถูกเข้ารหัส

Trojan/backdoor ใช้ศึกษาการรันคำสั่งและการคงอยู่บนเครื่อง ฝั่ง Linux มีการวาง backdoor เรียก shell และตั้งค่า cron กับ systemd ส่วน Windows ใช้ชุดทดสอบ PowerShell, Registry Run Keys และ Scheduled Task จึงศึกษาการสร้างกระบวนการไฟล์ที่ถูกวาง และกิจกรรม registry ของ Windows ได้ กลุ่มนี้เน้นพฤติกรรมหลังเริ่มทำงาน โดยไม่ได้ทดสอบการหลอกให้ผู้ใช้เปิดโปรแกรมซึ่งเป็นอีกด้านหนึ่งของ Trojan

Botnet ใช้ศึกษาการสื่อสารกับเซิร์ฟเวอร์ Command-and-Control ภายในแล็บ สคริปต์ติดต่อ HTTP/TCP เข้าและเตรียมข้อมูลสำหรับส่งออก EventID 3 เชื่อมการติดต่อกับกระบวนการต้นทาง จึงนำจำนวนและสัดส่วน NetworkConnect กับจำนวน IP และพอร์ตปลายทางที่ต่างกันมาศึกษาได้ การวิเคราะห์แบบ host-based ในกรณีนี้จึงอาศัยบริบทของโปรแกรมที่ติดต่อเครือข่าย ไม่ได้ยืนยันจากพอร์ตเพียงค่าเดียวว่าเป็น botnet

Cryptominer ใช้ศึกษาบริบทการใช้เครื่องเพื่อประมวลผลงานชุด ฝั่ง Linux รัน XMRig กับ pool ภายในแล็บ ส่วน Windows สร้างงานคำนวณและเชื่อมต่อ pool จำลอง ร่องรอยที่คุณลักษณะปัจจุบันใช้ได้คือการเริ่มและจบกระบวนการช่วงเวลาที่เกิด และการติดต่อปลายทาง ชุดข้อมูลและคุณลักษณะ 32 ตัวไม่ได้วัดเปอร์เซ็นต์ CPU หรือ GPU โดยตรง จึงยังไม่รองรับข้อสรุปว่าโมเดลตรวจการใช้ CPU สูงหรือยืนยันการชุดได้จาก Sysmon เพียงอย่างเดียว

Exploitation ใช้ศึกษาการยกระดับสิทธิ์และกิจกรรมหลังความพยายามโจมตี ฝั่ง Linux มีชุดทดสอบช่องโหว่และการรันโปรแกรมหลังยกระดับสิทธิ์ ส่วน Windows ทดสอบ UAC bypass, token manipulation และ process injection ร่องรอยที่ศึกษาได้ประกอบด้วยการสร้างกระบวนการ คำสั่ง ไฟล์ และกิจกรรม registry ที่รองรับ อย่างไรก็ตาม การมีเหตุการณ์

เหล่านี้ไม่ยืนยันว่าการยกระดับสิทธิ์สำเร็จ และขอบเขตข้อมูล CreateRemoteThread ยังจำกัดตามหัวข้อ 4.5 ชื่อ exploitation เป็นกลุ่มเทคนิคโจมตี ไม่ใช่ตระกูลมัลแวร์

ตาราง 1 ด้านพฤติกรรมและเหตุการณ์บนโฮสต์ที่ใช้ศึกษาสถานการณ์

สถานการณ์	ด้านที่ศึกษา	EventID ที่เกี่ยวข้อง
Benign	กิจกรรมผู้ใช้ปกติ	1, 3, 5, 11, 23
Ransomware	ผลกระทบต่อนโยบาย	1, 11, 23
Trojan/backdoor	รันคำสั่ง คงอยู่ในระบบ	1, 3, 11, 23 12, 13 (Windows)
Botnet	การสื่อสาร C2	1, 3, 11
Cryptominer	บริบทงานชุด การติดต่อ pool	1, 3, 5, 11
Exploitation	สิทธิ์และรันโค้ด	1, 11 8, 12, 13 (Windows)

ตาราง 1 สรุปเหตุการณ์ที่เกี่ยวข้องตามชนิดที่รองรับและการตั้งค่าการเก็บบันทึก [1] พฤติกรรมแต่ละกลุ่มอาจซ้อนทับกัน เช่น Trojan และ Botnet อาจติดต่อ C2 หรือมีการคงอยู่ในระบบเช่นเดียวกัน กลุ่มทั้งห้าจึงเป็นบริบทการสร้างและประเมินข้อมูล ส่วนแบบจำลองหลักจำแนกสองป้ายกำกับ คือกิจกรรมปกติและกิจกรรมที่เชื่อมโยงกับสถานการณ์โจมตี การมี EventID ใดรายการหนึ่งยังไม่เพียงพอที่จะระบุประเภทมัลแวร์

3.3 การประมวลผลและการกำหนดป้ายกำกับ

บันทึกของทั้งสองแพลตฟอร์มผ่านขั้นแยกวิเคราะห์ เติมข้อมูล และกำหนดป้ายกำกับ ขั้นแยกวิเคราะห์แปลงข้อมูลเหตุการณ์เป็นตาราง ขันเติมข้อมูลเชื่อมข้อมูลที่เกี่ยวข้องกับกระบวนการ เช่น บรรทัดคำสั่งและข้อมูลกระบวนการแม่ ก่อนรวมข้อมูลจากทุกรอบเป็นไฟล์เดียว คอลัมน์ที่ระบุเครื่องแพลตฟอร์ม และรอบทดลองเก็บไว้สำหรับตรวจสอบที่มาและกำหนดกลุ่มประเมิน

ในรอบสถานการณ์โจมตี กำหนดกระบวนการเริ่มต้นจากตำแหน่งไฟล์หรือคำสั่งของชุดทดสอบ แล้วติดตามกระบวนการลูกที่สืบสายจากกระบวนการเริ่มต้น เหตุการณ์ที่เชื่อมโยงได้กำหนด label=1 ส่วนเหตุการณ์ที่ไม่อยู่ในสายดังกล่าวกำหนด label=0 รอบพฤติกรรมปกติกำหนด label=0 ตามรอบทดลอง ในข้อมูลรวมมี 62,035 เหตุการณ์ที่ใช้วิธี lineage และ 19,296

เหตุการณ์ที่ใช้วิธี session โดยเหตุการณ์จากวิธี session ทั้งหมดมี label=0

ป้ายกำกับอันตรายในงานนี้หมายถึงความสัมพันธ์กับกระบวนการทดลองโจมตี การมี label=1 จึงไม่ได้หมายความว่าทุกเหตุการณ์เป็นการกระทำอันตรายในตัวเอง คอลัมน์ label_method และ is_seed ช่วยตรวจสอบกติกาที่ใช้กำหนดป้ายกำกับ ความถูกต้องของป้ายกำกับยังต้องตรวจสอบร่วมกับรายละเอียดการทำงานของชุดทดสอบ

3.4 การสร้างคุณลักษณะระดับกระบวนการ

การรวมข้อมูลปัจจุบันใช้ ProcessGuid เป็นกุญแจและตัดแถวที่ไม่มีค่านี้อีกก่อนสร้างเวกเตอร์ ได้ 23,971 แถว แต่ละแถวใช้ label สูงสุดของเหตุการณ์ในกลุ่มเป็นป้ายกำกับ กระบวนการที่มีเหตุการณ์ label=1 อย่างน้อยหนึ่งรายการจึงมีป้ายกำกับเป็น 1 ข้อจำกัดของการใช้ GUID เพียงคอลัมน์เดียวและการจัดการ GUID ศูนย์อธิบายในหัวข้อ 5.2

สร้างคุณลักษณะเริ่มต้น 35 ตัว ได้แก่ จำนวนเหตุการณ์ จำนวนและสัดส่วนเหตุการณ์แต่ละชนิด จำนวนค่าปลายทางที่ต่างกัน ระยะเวลาที่สังเกต อัตราเหตุการณ์ ความยาวคำสั่งและเส้นทางโปรแกรม รวมถึงข้อมูลจากขั้นเติมข้อมูล ผลหลักตัด img_len, depth และ enriched ออก เหลือ 32 ตัวตามตาราง 2 ชุดคุณลักษณะนี้ไม่มีจำนวนกระบวนการลูกหรือค่า IntegrityLevel โดยตรง การสืบสายใช้ในขั้นกำหนดป้ายกำกับ และไม่ได้เป็นคุณลักษณะกราฟที่ป้อนโมเดล

ตาราง 2 องค์ประกอบของคุณลักษณะระดับกระบวนการ 32 ตัว

กลุ่ม	คุณลักษณะ	จำนวน
ปริมาณ	n_events	1
ชนิดเหตุการณ์	ev_ID และ frac_ID สำหรับ EventID 11 ชนิด	22
ค่าที่แตกต่าง	n_files, n_dst_ip, n_dst_port, n_regobj, n_device	5
เวลาและอัตรา	dur_s และ rate	2
คำสั่ง	cmd_len และ pcmd_len	2
รวม		32

สถิติระยะเวลาคำนวณจากเวลาสุดท้ายลบเวลาแรกในกลุ่ม และอัตราเหตุการณ์หารด้วยระยะเวลาอย่างน้อยหนึ่งวินาที ค่าที่ไม่มีข้อมูลถูกเติมเป็น 0 ตามกฎปัจจุบัน ส่วนความยาวคำสั่ง

คำนวณจากค่าคำสั่งแรกที่มีในกลุ่ม การเติม 0 ยังไม่แยกระหว่างการไม่เกิดเหตุการณ์กับการไม่รองรับเหตุการณ์ของแพลตฟอร์ม จึงต้องคำนึงถึงข้อจำกัดนี้เมื่อตีความคุณลักษณะ

3.5 แบบจำลองและการเตรียมข้อมูล

พัฒนาและประเมินแบบจำลองด้วย scikit-learn [13] ผลหลักใช้คุณลักษณะทั้ง 32 ตัวโดยไม่ใช้ PCA และปรับมาตรฐานด้วย StandardScaler ซึ่งเรียนรู้ค่าจากชุดฝึกภายนอกแล้วใช้การแปลงเดียวกันกับชุดทดสอบ แบบมีผู้สอนประกอบด้วย Naive Bayes, Decision Tree, Random Forest [14] และ SVM [15] โดย Decision Tree, Random Forest และ SVM ใช้ class_weight แบบ balanced ส่วน Naive Bayes ใช้ GaussianNB

กำหนดพารามิเตอร์คงที่สำหรับการทดลองหลัก Random Forest ใช้ต้นไม้ 70 ต้น min_samples_leaf=9, max_features=sqrt และ bootstrap=False; Decision Tree ใช้ criterion=entropy, max_depth=17, min_samples_split=14, min_samples_leaf=5 และ max_features=log2; SVM ใช้ RBF kernel, C=10 และ gamma=scale; GaussianNB ใช้ var_smoothing=10 ยกกำลัง -8 ค่าดังกล่าวเป็น configuration ของผลที่รายงาน ไม่ใช่ผลค้นหาพารามิเตอร์ที่ดีที่สุดบนชุดข้อมูลนี้

แบบไม่มีผู้สอนประกอบด้วย Isolation Forest [16], Local Outlier Factor [17] และ One-Class SVM [18] ซึ่งฝึกด้วยกระบวนการ label=0 เท่านั้น Isolation Forest ใช้ต้นไม้ 200 ต้น; Local Outlier Factor ใช้เพื่อนบ้าน 20 ตัวและ novelty=True; One-Class SVM ใช้ RBF kernel และตัวอย่างฝึกปกติไม่เกิน 6,000 แถว การทดลองหลักเริ่มด้วย contamination หรือ nu เท่ากับ 0.1 และกำหนด random seed=0 ในขั้นที่รองรับ

สำหรับการเลือกค่าตัด แบ่งข้อมูลฝึกภายนอกเป็นส่วนย่อย 75:25 ตามกระบวนการ แล้วเลือกสัดส่วน quantile ที่ให้ F1 สูงสุดบนส่วน validation โดยกวาดตั้งแต่ 0.05 ถึง 0.60 ครั้งละ 0.05 ผลหลักเลือกสัดส่วน 0.45 และใช้ quantile ของคะแนนข้อมูลฝึกทั้งหมดเป็นค่าตัดสุดท้าย วิธีปัจจุบัน fit scaler และโมเดลไม่มีผู้สอนก่อนแยกคะแนน validation จึงมีข้อมูลปกติ

จากส่วน validation อยู่ในการ fit ด้วย ผลส่วนนี้ควรตีความเป็นผลเบื้องต้น และต้องยืนยันด้วยชุด calibration ที่เป็นอิสระ

3.6 การแบ่งข้อมูลและตัวชี้วัด

ผลหลักใช้ GroupShuffleSplit ตาม ProcessGuid หนึ่งครั้ง สัดส่วนฝึกต่อทดสอบ 70:30 และ random seed=0 ได้ชุดฝึก 16,779 แถวและชุดทดสอบ 7,192 แถว โดยมี label=1 จำนวน 5,612 และ 2,399 แถวตามลำดับ ไม่มี ProcessGuid เดียวกันข้ามชุด การแบ่งนี้ลดการทดสอบด้วยเหตุการณ์จากกระบวนการเดียวกับที่ฝึก แต่ยังมีความสัมพันธ์ระหว่างกระบวนการจากสายหรือรอบทดลองเดียวกันได้

การประเมินเพิ่มเติมสร้างกลุ่มจากแพลตฟอร์มร่วมกับชื่อสถานการณ์ และกันกลุ่มโจมตีออกทีละกลุ่มรวม 10 กลุ่ม โดยไม่กันกลุ่ม benign ออก ตัวอย่างเช่น เมื่อกัน ransomware บน Linux ออก ชุดฝึกยังมี ransomware บน Windows การประเมินนี้จึงศึกษาคู่แพลตฟอร์ม-สถานการณ์ที่กันออก ภายในแล็บ ไม่ใช่การกันการโจมตีชนิดเดียวกันออกจากทั้งสองแพลตฟอร์ม ในการประเมินนี้ โมเดลไม่มีผู้สอนใช้ contamination หรือ nu เท่ากับ 0.3 โดยไม่ปรับค่าตัดแบบการทดลองหลัก

รายงาน Accuracy, Precision, Recall, F1 และ ROC-AUC โดยกำหนด label=1 เป็นคลาสบวก F1 เป็นค่าเฉลี่ยฮาร์โมนิกของ Precision และ Recall ส่วน ROC-AUC คำนวณจากคะแนนต่อเนื่องของแบบจำลอง นอกจากนี้ใช้วิธีแจ้งเตือนทุกแถวเป็นเส้นฐานภายใน เพื่อให้เห็นว่าค่า Recall สูงเพียงอย่างเดียวเพียงพอต่อการจำแนกหรือไม่ ผลการกันกลุ่มออกสรุปเป็นค่าเฉลี่ยรายกลุ่ม และนับจำนวนกลุ่มที่ F1 สูงกว่าวิธีแจ้งเตือนทุกแถว

4. การทดลองและผลลัพธ์

4.1 องค์ประกอบของชุดข้อมูล

ชุดข้อมูลดิบมี 81,331 เหตุการณ์ 80 คอลัมน์ แบ่งเป็น Linux 45,311 และ Windows 36,020 เหตุการณ์ มี label=1 รวม 27,499 เหตุการณ์ หรือ 33.8% องค์ประกอบแสดงในตาราง 3 สัดส่วน label=1 ระดับเหตุการณ์ของ Linux และ Windows เท่ากับ 34.2% และ 33.3% ตามลำดับ

ตาราง 3 จำนวนเหตุการณ์จำแนกตามแพลตฟอร์มและป้ายกำกับ

แพลตฟอร์ม	ปกติ	อันตราย	รวม	%อันตราย
Linux	29,805	15,506	45,311	34.2%
Windows	24,027	11,993	36,020	33.3%
รวม	53,832	27,499	81,331	33.8%

เมื่อรวมข้อมูลตามกฎระดับกระบวนการปัจจุบัน มี label=0 จำนวน 15,960 แถวและ label=1 จำนวน 8,011 แถว กระบวนการที่จัดอยู่ใน Linux มีสัดส่วน label=1 ประมาณ 30.6% ส่วน Windows ประมาณ 42.5% ดังนั้นความใกล้เคียงของสัดส่วนระดับเหตุการณ์ไม่สามารถใช้ยืนยันความสมดุลระดับกระบวนการได้ และต้องรายงานหน่วยตัวอย่างร่วมกับสัดส่วนคลาส

4.2 ผลการจำแนกระดับกระบวนการ

ตาราง 4 รายงานผลจาก process holdout หลัก Random Forest ให้ผลสูงสุดในกลุ่มมีผู้สอน โดยมี Accuracy 0.949, Precision 0.919, Recall 0.929, F1 0.924 และ ROC-AUC 0.988 ส่วน Decision Tree ให้ F1 0.804 SVM ให้ 0.607 และ Naive Bayes ให้ 0.505 ภาพ 2 แสดง Precision, Recall และ F1 จากผลทดลองชุดเดียวกับตาราง

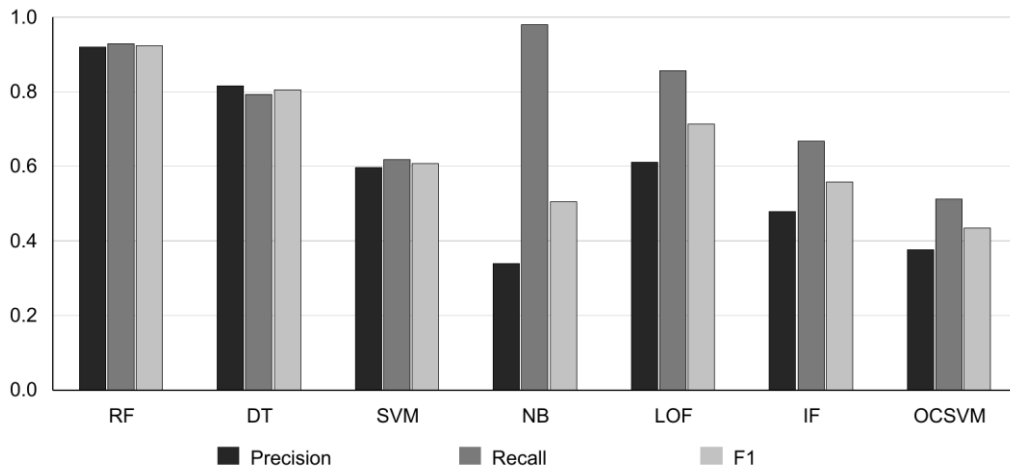
ตาราง 4 ผล process holdout หนึ่งครั้ง ใช้ 32 คุณลักษณะและไม่ใช้ PCA

แบบจำลอง	Acc	Prec	Rec	F1	AUC
Random Forest	0.949	0.919	0.929	0.924	0.988
Decision Tree	0.871	0.817	0.792	0.804	0.930
SVM	0.733	0.597	0.619	0.607	0.786
Naive Bayes	0.359	0.340	0.980	0.505	0.594
Local Outlier Factor	0.771	0.612	0.857	0.714	0.849
Isolation Forest	0.647	0.479	0.667	0.558	0.622
One-Class SVM	0.554	0.376	0.513	0.434	0.499
แจ้งเตือนทุกแถว	0.334	0.334	1.000	0.500	-

ในกลุ่มไม่มีผู้สอน Local Outlier Factor ให้ F1 0.714 สูงกว่าวิธีแจ้งเตือนทุกแถวที่ F1 0.500 และมี ROC-AUC 0.849 ส่วน Isolation Forest และ One-Class SVM ให้ F1 0.558 และ 0.434 ตามลำดับ Naive Bayes มี Recall 0.980 แต่ Precision 0.340 จึงแสดงว่าการตรวจพบคลาสบวกจำนวนมาก

ต้องพิจารณา ร่วมกับการแจ้งเตือนผิด ผลของโมเดลไม่มีผู้สอนยัง
ขึ้นกับวิธีเลือกค่าตัดที่อธิบายในหัวข้อ 3.5

Precision, Recall and F1 on the main process holdout



ภาพ 2 Precision Recall และ F1 ของแบบจำลองบน process holdout หลัก

4.3 ผลการกันคู่แพลตฟอร์มและสถานการณ์ออก

การกันคู่แพลตฟอร์ม-สถานการณ์โจมตีออกทีละกลุ่มให้ค่าเฉลี่ยของ Random Forest เท่ากับ F1 0.764 และ ROC-AUC 0.897 โดยมี F1 สูงกว่าวิธีแจ้งเตือนทุกแถว 8 จาก 10 กลุ่ม Local Outlier Factor ให้ F1 เฉลี่ย 0.728 และ ROC-AUC เฉลี่ย 0.780 และชนะเส้นฐานภายในครบ 10 กลุ่ม ดังตาราง 5 การนับกลุ่มที่ชนะช่วยอธิบายผลรวมกับค่าเฉลี่ย แต่ไม่ได้แทนการรายงานจำนวนการแจ้งเตือนผิดของแต่ละกลุ่ม ตาราง 5 ค่าเฉลี่ยรายกลุ่มจากการกันคู่แพลตฟอร์มและสถานการณ์ออก 10 กลุ่ม

แบบจำลอง	F1 เฉลี่ย	AUC เฉลี่ย	ชนะเส้นฐาน
Random Forest	0.764	0.897	8/10
Local Outlier Factor	0.728	0.780	10/10
Decision Tree	0.688	0.820	7/10

ค่า F1 เฉลี่ยของ Random Forest ในการกันกลุ่มออกต่ำกว่าผล process holdout แสดงว่าผลการทำนายขึ้นกับองค์ประกอบของชุดฝึกและชุดทดสอบ การทดลองนี้ยังมีสถานการณ์ประเภทเดียวกันจากอีกแพลตฟอร์มอยู่ในการฝึกและใช้รอบเก็บจากแล็บเดิม ผลจึงรองรับการอธิบายการทำนายกลุ่มที่กันออกภายใต้เงื่อนไขดังกล่าว และยังไม่ใช่หลักฐานของการตรวจจับมัลแวร์ zero-day จริง

4.4 ความอ่อนไหวต่อวิธีแบ่งข้อมูลระดับเหตุการณ์

การทดลองเพิ่มเติมระดับเหตุการณ์ใช้คุณลักษณะ 48 ตัว ลดมิติด้วย PCA เหลือ 9 องค์ประกอบ และใช้ Random Forest 150 ต้น เพื่อศึกษาความอ่อนไหวต่อวิธีแบ่งข้อมูล ผลในตาราง 6 มาจาก configuration นี้ทั้งหมด ซึ่งเป็นการทดลองแยกจากผลระดับกระบวนการในตาราง 4

ตาราง 6 RF ระดับเหตุการณ์ 48 คุณลักษณะ PCA 9 องค์ประกอบ

การแบ่ง holdout	Accuracy	F1
สุ่มรายเหตุการณ์	0.974	0.961
จัดกลุ่มตาม ProcessGuid	0.881	0.811
จัดกลุ่มแพลตฟอร์มและสถานการณ์	0.636	0.634

การสุ่มรายเหตุการณ์ให้ F1 0.961 การแบ่งตาม ProcessGuid ให้ 0.811 และการแบ่งตามคู่แพลตฟอร์ม-สถานการณ์ให้ 0.634 ทั้งสองการแบ่งแบบจัดกลุ่มใช้ GroupShuffleSplit หนึ่งครั้ง โดยแถวสุดท้ายไม่ได้กันออกทีละกลุ่มจนครบ อัตรา label=1 ใน test เท่ากับ 33.11%, 27.14% และ 40.09% ตามลำดับ ผลจึงแสดงความอ่อนไหวต่อการแบ่งและองค์ประกอบ test ที่เปลี่ยนไป ไม่ใช่ค่าประมาณผลของ leakage เพียงปัจจัยเดียว

4.5 การกระจายชนิดเหตุการณ์

ข้อมูลดิบประกอบด้วยเหตุการณ์ 11 ชนิดตามตาราง 7 ProcessTerminate และ ProcessCreate มีสัดส่วนมากที่สุด ส่วน NetworkConnect มี 2,282 เหตุการณ์ และ FileDelete มี 7,780 เหตุการณ์ องค์ประกอบนี้ช่วยระบุว่ากิจกรรมใดมีข้อมูลสำหรับศึกษา แต่จำนวนเหตุการณ์เพียงอย่างเดียวยังไม่บอกประสิทธิภาพการตรวจจับในแต่ละสถานการณ์

ตาราง 7 จำนวนและสัดส่วนชนิดเหตุการณ์ในชุดข้อมูลดิบ

ID	เหตุการณ์	จำนวน	%
1	ProcessCreate	23,317	28.67
2	FileCreateTime	1,743	2.14
3	NetworkConnect	2,282	2.81
4	SysmonStateChange	6	0.007
5	ProcessTerminate	26,863	33.03
8	CreateRemoteThread	2	0.002
9	RawAccessRead	4,998	6.15
11	FileCreate	9,534	11.72
12	RegistryCreateDelete	117	0.14
13	RegistrySetValue	4,689	5.77
23	FileDelete	7,780	9.57

CreateRemoteThread มีเพียงสองเหตุการณ์ และทั้งสองไม่มี ProcessGuid จึงถูกตัดก่อนรวมระดับกระบวนการ ข้อจำกัดนี้ทำให้ผลจำแนกระดับกระบวนการยังไม่รองรับข้อสรุปเฉพาะการแทรกโค้ดอย่างเพียงพอ การเก็บข้อมูลเพิ่มเติมควรตรวจการระบุกระบวนการต้นทางและปลายทาง รวมถึงความครอบคลุมของแต่ละชนิดเหตุการณ์

5. อภิปรายผลและข้อจำกัด

5.1 การตีความผลภายในแล็บ

ผล process holdout แสดงว่าเวกเตอร์สถิติที่สร้างจากบันทึก Sysmon มีข้อมูลสำหรับแยกป้ายกำกับภายในแล็บนี้ Random Forest ให้ Precision และ Recall ใกล้เคียงกัน ขณะที่ Local Outlier Factor จำแนกจากรูปแบบข้อมูลปกติได้โดยไม่ใช้ label=1 ในการฝึก อย่างไรก็ตาม ข้อมูลมาจากสถานการณ์จำลองและพฤติกรรมปกติที่จำกัด ความยาวคำสั่งหรือกิจกรรม

ของชุดทดสอบอาจมีรูปแบบเฉพาะ จึงยังต้องประเมินว่าสัญญาณที่แบบจำลองใช้คงอยู่ในโปรแกรมและรอบทดลองอื่นหรือไม่

การใช้ข้อมูลสองแพลตฟอร์มทำให้ชุดข้อมูลมีบริบทของกิจกรรมหลายระบบปฏิบัติการ แต่ไม่ได้รับรองว่าแบบจำลองเป็นอิสระจากแพลตฟอร์ม สัดส่วนคลาสระดับกระบวนการและรูปแบบข้อมูลที่แตกต่างกันระหว่างสองระบบ การยืนยันความสามารถข้ามแพลตฟอร์มต้องออกแบบการฝึกและทดสอบแยกตามแพลตฟอร์ม พร้อมควบคุมประเภทสถานการณ์และหน่วยตัวอย่าง ส่วนผลความอ่อนไหวในตาราง 6 สนับสนุนให้รายงานวิธีแบ่งข้อมูลทุกครั้ง และไม่ได้พิสูจน์ว่าวิธีใดมีผลมากกว่าปริมาณข้อมูล

5.2 ข้อจำกัดของข้อมูลและการประเมิน

ข้อมูลมีหนึ่งรอบต่อสถานการณ์ต่อแพลตฟอร์มและมีเครื่องเป้าหมายจำกัด การแบ่งตามกระบวนการยังอาจมีสายแม่-ลูกหรือกิจกรรมจากรอบเดียวกันอยู่ทั้งสองชุด ส่วนการกันคู่แพลตฟอร์ม-สถานการณ์ออกยังมีประเภทสถานการณ์เดียวกันจากอีกระบบอยู่ในชุดฝึก จึงต้องเพิ่มการทดสอบข้ามรอบ ข้ามเครื่อง และการกันสถานการณ์ชนิดเดียวกันออกจากทั้งสองระบบก่อนสรุปความสามารถต่อสภาพแวดล้อมที่ไม่เคยพบ

การรวมด้วย ProcessGuid เพียงคอลัมน์เดียวมีข้อจำกัดเมื่อ GUID ซ้ำระหว่างรอบ ในข้อมูลพบ GUID ศูนย์จำนวน 612 เหตุการณ์กระจายหลายรอบ และมี GUID จริงหนึ่งค่าซ้ำใน Linux exploit และ miner รวม 12 เหตุการณ์ นอกจากนี้แปดเหตุการณ์ไม่มี GUID การแก้ไขรอบถัดไปควรใช้กฎแฉ่ร่วมของแพลตฟอร์ม เครื่อง รอบเก็บ และ GUID พร้อมกำหนดวิธีจัดการ GUID ศูนย์และเหตุการณ์ที่ใช้ SourceProcessGuid หรือ TargetProcessGuid แล้วสร้างคุณลักษณะและประเมินใหม่

การเลือกค่าตัดของโมเดลไม่มีผู้สอนใช้ส่วน validation ที่มีข้อมูลปกติรวมกับการ fit และค่าตัดสุดท้ายอ้าง quantile ของคะแนน train ต่างจาก quantile ที่ใช้เลือกบน validation ผลด้านนี้จึงต้องยืนยันด้วยการแยก fit และ calibration ก่อนเรียนรู้ preprocessing และโมเดล โดยตรงค่าตัดก่อนทดสอบรวมทั้งรายงาน confusion matrix และอัตราการแจ้งเตือนผิดพลาดหลักปัจจุบันเป็นผลจากการแบ่งครั้งเดียว จึงยังไม่มีช่วงความเชื่อมั่นจากรอบเก็บอิสระ

ความครบถ้วนของ lineage และกฎกำหนดป้ายกำกับมีผลต่อความหมายของคลาส การติดตามลูกหลานยืนยันความเชื่อมโยงกับการทดลองได้ แต่ควรตรวจตัวอย่างเหตุการณ์ร่วมกับหลักฐานจากชุดทดสอบ นอกจากนี้ต้องตรวจการแทนค่าคำสั่งที่หายไป การรองรับชนิดเหตุการณ์ของแต่ละแพลตฟอร์มและผลของการปิด Defender ต้องประกอบข้อมูล เพื่อให้การประเมินภายหลังสะท้อนการใช้งานที่ต้องการ

6. สรุปและงานในอนาคต

งานนี้สร้างชุดข้อมูล Sysmon จาก Linux และ Windows จำนวน 81,331 เหตุการณ์ พร้อมกฎกำหนดป้ายกำกับและคุณลักษณะระดับกระบวนการ 32 ตัว ผลเบื้องต้นจาก process holdout หนึ่งครั้งให้ Random Forest มี Accuracy 94.9% และ F1 0.924 ส่วน Local Outlier Factor ให้ F1 0.714 การประเมินโดยกันคู่แพลตฟอร์ม-สถานการณ์ออกให้ F1 เฉลี่ยของ Random Forest 0.764 ผลทั้งหมดควรตีความภายในเงื่อนไขการเก็บและการประเมินที่ระบุ

งานต่อไปควรแก้ปัญหาแฉรวมกระบวนการ แยก fit และ calibration ให้เป็นอิสระ เก็บแต่ละสถานการณ์ซ้ำจากหลายเครื่อง และเพิ่มความหลากหลายของกิจกรรมปกติกับมัลแวร์จริง จากนั้นประเมินข้ามรอบและข้ามแพลตฟอร์ม พร้อมวัดภาระการแจ้งเตือนผิด เพื่อให้เห็นว่รูปแบบที่เรียนรู้จากแล็บสามารถนำไปใช้กับข้อมูลใหม่ได้เพียงใด

เอกสารอ้างอิง

- [1] M. Russinovich and T. Garnier, "Sysmon (System Monitor)," Microsoft Sysinternals, 2024. [Online]. Available: <https://learn.microsoft.com/sysinternals/downloads/sysmon>
- [2] D. Arp et al., "Dos and Don'ts of Machine Learning in Computer Security," in Proc. 31st USENIX Security Symposium, 2022, pp. 3971-3988.
- [3] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time," in Proc. 28th USENIX Security Symposium, 2019, pp. 729-746.
- [4] B. E. Strom et al., "MITRE ATT&CK: Design and Philosophy," The MITRE Corporation, 2020.
- [5] Red Canary, "Atomic Red Team," GitHub repository, 2024. [Online]. Available: <https://github.com/redcanaryco/atomic-red-team>
- [6] R. V. Mahmoud, M. Anagnostopoulos, S. Pastrana, and J. M. Pedersen, "Redefining Malware Sandboxing: Enhancing Analysis Through Sysmon and ELK Integration," IEEE Access, vol. 12, pp. 68624-68636, 2024.
- [7] B. A. Pratomo, S. A. Kosim, H. Studiawan, and A. O. Prabowo, "BarongTrace: A Malware Event Log Dataset for Linux," in Advanced Information Networking and Applications, LNDECT, vol. 202, pp. 48-60, 2024.
- [8] D. Sgandurra, L. Munoz-Gonzalez, R. Mohsen, and E. C. Lupu, "Automated Dynamic Analysis of Ransomware: Benefits, Limitations and use for Detection," arXiv:1609.03020, 2016.
- [9] J. A. Herrera-Silva and M. Hernandez-Alvarez, "Dynamic Feature Dataset for Ransomware Detection Using Machine Learning Algorithms," Sensors, vol. 23, no. 3, 1053, 2023.
- [10] B. A. Pratomo et al., "Enhancing Enterprise Network Security: Comparing Machine-Level and Process-Level Analysis for Dynamic Malware Detection," arXiv:2310.18165, 2023.
- [11] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime Provenance-Based Detector for Advanced Persistent Threats," in Proc. NDSS Symposium, 2020.

- [12] Z. Cheng et al., "Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance," in Proc. IEEE Symposium on Security and Privacy, 2024.
- [13] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [14] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.
- [15] C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273-297, 1995.
- [16] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation Forest," in Proc. IEEE ICDM, 2008, pp. 413-422.
- [17] M. M. Breunig, H. P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying Density-Based Local Outliers," in Proc. ACM SIGMOD, 2000, pp. 93-104.
- [18] B. Scholkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the Support of a High-Dimensional Distribution," Neural Computation, vol. 13, no. 7, pp. 1443-1471, 2001.